

CS633 Assignment Group Number 38

Arfa Khayal(251110013) Divakar Bajpai(251110019) Nishant(251110053)
Shubham Rawat(251110070) Pujari Krishnachandu (220832)

1 Broad Approach

- Each process counts isovalue once (no double count), Isovalue count on edges (no diagonal count)
- We start with laying out process ranks in a 3D cubic fashion with dimensions p_x , p_y , p_z . Each process is responsible for a local 3D block of size $n_x \times n_y \times n_z$. We identify neighbours based on calculated stencil radius and use array for storing neighboring process ranks in each direction (left, right, up, down, front, back). Derived MPI vector datatype is used for efficient halo exchange in x, y, and z directions. The main computation loop consists of T iterations. We first perform halo exchange with neighbors in all 6 directions (if they exist), we then compute the new values for the interior cells based on the stencil operation. We count isovalues unidirectionally in x, y, and z directions by detecting sign change for each field and use MPI_Reduce to get the global counts. Finally, we print the execution time.

2 Code Description

- **Process Mapping**

- Map each rank to its 3D coordinates: $m_x = \text{rank} \bmod p_x$, $m_y = \left(\frac{\text{rank}}{p_x}\right) \bmod p_y$, $m_z = \frac{\text{rank}}{p_x \cdot p_y}$
- Re-arrange processes in a $p_x \times p_y \times p_z$ grid

- **Domain Decomposition**

- Each process handles a local grid of size $n_x \times n_y \times n_z$
- Add halo layers required stencil distance based on given d point stencil :

$$\text{stencil radius} = \frac{d-1}{6}$$

- **Neighbor Identification**

- Neighbors: Left $(m_x - 1, m_y, m_z)$, Right $(m_x + 1, m_y, m_z)$, Down $(m_x, m_y - 1, m_z)$, Up $(m_x, m_y + 1, m_z)$, Back $(m_x, m_y, m_z - 1)$, Front $(m_x, m_y, m_z + 1)$
- Mark boundaries where neighbors do not exist

- **Time Iteration ($t = 0$ to T)**

- **Halo Exchange**

- * Uses MPI_Isend, MPI_Irecv with derived data type MPI_Type_vector for efficient halo exchange.

- **Defining Derived Datatypes (MPI_Type_vector)**

- * Used to communicate non-contiguous boundary data (halo regions) efficiently.
- * X-direction (YZ planes): MPI_Type_vector(ny, nz, LZ, MPI_DOUBLE)
- * Y-direction (XZ planes): MPI_Type_vector(nx, nz, LY*LZ, MPI_DOUBLE)
- * Z-direction (halo strips): MPI_Type_vector(ny, r, LZ, MPI_DOUBLE)

– **Isovalue Counting (for $t > 0$)**

- * Initialize local count = 0
- * For each interior cell, check sign change with forward neighbors:

$$(v_i - iso)(v_j - iso) < 0$$

- * Isovalue crossings counted only in forward directions (+X, +Y, +Z) that eliminates redundant computations
- * Perform `MPI_Reduce` to obtain total count

– **Stencil Computation (for $t < T$)**

- * For each interior cell:
 - Initialize sum with current value
 - Add neighbors in $\pm X, \pm Y, \pm Z$ up to radius r
 - Identify existing neighbours and compute average over actual number of valid neighbors:

$$\text{new value} = \frac{\text{sum}}{\text{count}}$$

- * Store results in a separate array

• **Buffer Update**

- Swap current and next arrays after each timestep

• **Output**

- Root process prints isovalue counts after each timestep and total execution time.

3 Code Optimizations

• **Non-blocking Communication**

- Used `MPI_Isend` and `MPI_Irecv` to overlap communication and computation that reduces idle waiting time compared to blocking communication

• **Derived Datatypes**

- Used `MPI_Type_vector` for sending non-contiguous data (planes/lines)
- Avoids manual packing/unpacking of data
- Improves communication efficiency

• **Buffer Swapping**

- Swapping pointers instead of copying arrays
- Reduces memory overhead and improves performance

• **Efficient Reduction**

- Used `MPI_Reduce` to aggregate results efficiently

Processes (P)	Grid ($p_x \times p_y \times p_z$)	Avg Time (120^3) (s)	Avg Time (240^3) (s)
32	$4 \times 4 \times 2$	1.5242	10.0711
48	$6 \times 4 \times 2$	1.8245	11.8686
64	$4 \times 4 \times 4$	1.8661	12.1070
96	$6 \times 4 \times 4$	1.9236	12.2250

Table 1: Comparison of Average Execution Time for Different Data Sizes per process using the **Naïve Implementation**

Processes (P)	Grid ($p_x \times p_y \times p_z$)	Avg Time (120^3) (s)	Avg Time (240^3) (s)
32	$4 \times 4 \times 2$	0.5170	4.4589
48	$6 \times 4 \times 2$	0.5815	5.2186
64	$4 \times 4 \times 4$	0.5890	5.1720
96	$6 \times 4 \times 4$	0.5952	5.1381

Table 2: Comparison of Average Execution Time for Different Data Sizes per process using the **Optimized Implementation**

4 Results

4.1 Execution Time Results

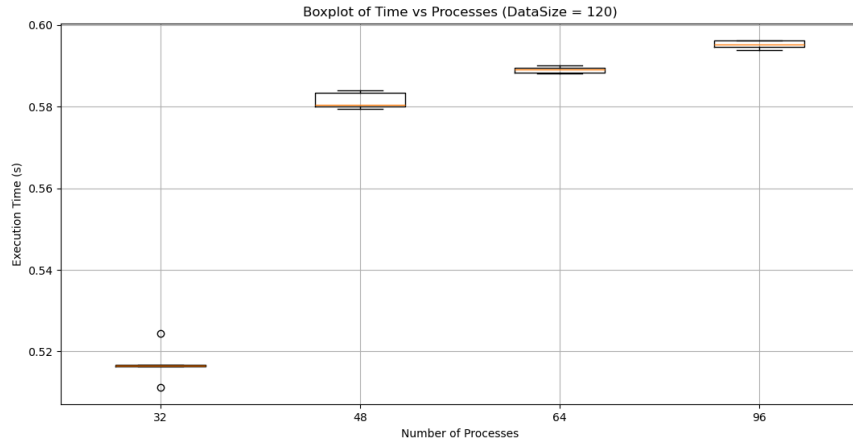


Figure 1: Boxplot of execution time vs number of processes for DataSize = 120.

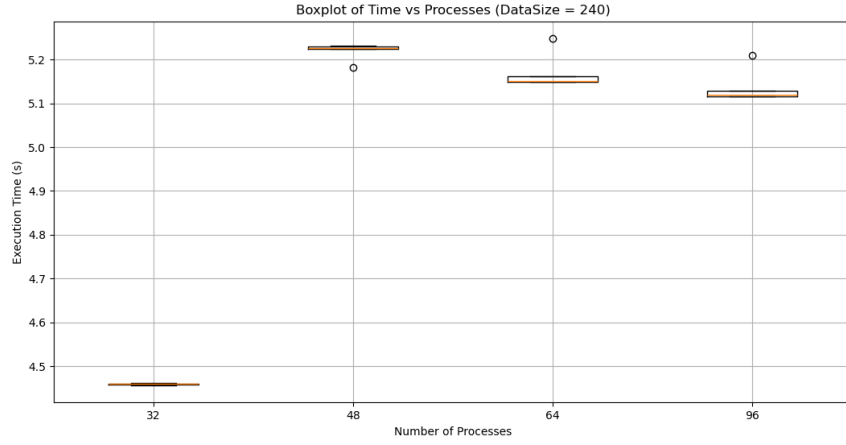


Figure 2: Boxplot of execution time vs number of processes for DataSize = 240.

5 Observations

- Execution time increases significantly with per process data size ($240 > 120$), due to higher computational workload.
- Increasing the number of processes does not improve performance; instead, execution time slightly increases due to communication overhead and synchronization costs outweighing the benefits of parallelism for experimented grid sizes.
- Communication overhead (halo exchange) dominates at higher process counts, limiting scalability.
- Smaller subdomain sizes per process lead to reduced computation and higher relative overhead.
- Parallel efficiency decreases as the number of processes increases.
- Low variation in execution time across runs indicates stable performance.

6 Conclusions

The MPI-based 3D stencil implementation correctly performs parallel computation with halo exchange and isovalue analysis. However, scalability is limited at higher process counts due to increasing communication overhead dominating computation.

All five group members contributed equally.